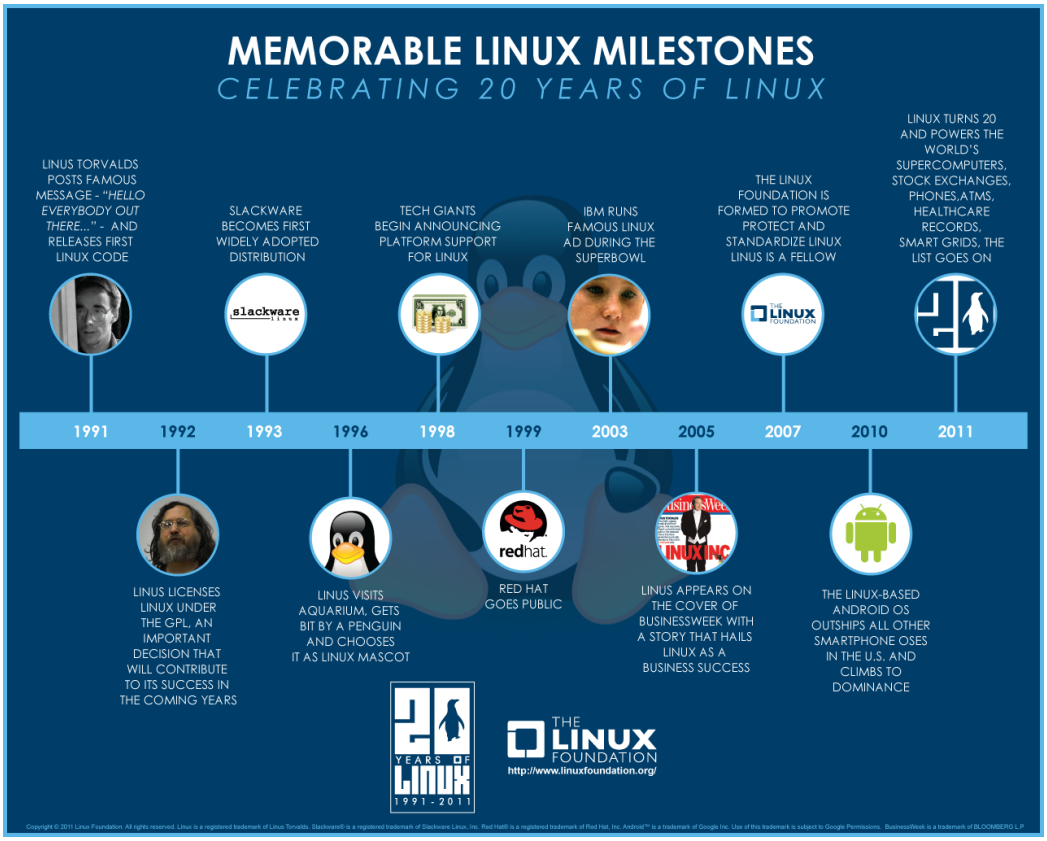
Module 1: What Is Linux?

Lesson 1.1: Overview of Linux

- **Lesson Objectives**
  - By the end of this lesson, you should be able to explain the origins of Linux, understand the open-source philosophy behind it, and recognize how Linux differs from other operating systems.



- **The Origins and History of Linux**

  Linux traces its roots to the Unix operating system developed at AT&T's Bell Labs in the 1960s. Unix introduced a multiuser, multitasking paradigm that would influence countless operating systems. In 1991, a Finnish computer science student named Linus Torvalds created a free, open-source version of the Unix kernel. He posted it online, inviting collaboration from programmers worldwide, and thus the Linux kernel was born.

  Because Torvalds released Linux under the GNU General Public License (GPL), the community was able to study, modify, and distribute its source code without paying licensing fees. Over time, contributions from thousands of individuals and

organizations expanded Linux into a robust kernel that, when combined with GNU software and other open-source tools, formed a complete operating system (often referred to as GNU/Linux).

- Key Milestones include Linux adopting the GPL, the rise of enterprise distributions in the early 2000s (such as Red Hat Enterprise Linux and SUSE), and today's vast ecosystem in which Linux runs on everything from servers and supercomputers to smartphones and embedded devices.

This video provides an engaging overview of Linux's inception and development. (Linux: The Origin Story)

Lesson 1.2: Common Linux Distributions and Installation Basics

- **Lesson Objectives**
  - Recognize the key differences among various Linux distributions.
  - Understand the basic installation process for a Linux distro, whether on physical hardware or in a virtual environment.
  - Identify which distro might best suit your needs based on features and use cases.

This video offers insights into various Linux distributions, aiding in understanding their unique attributes. (Comparing 10 Popular Linux Distros: Which One Rules Your Desktop?)

- **Introduction to Linux Distributions**

  A Linux distribution (often called a distro) is a packaged combination of the Linux kernel, system utilities, and additional software tailored to meet specific use cases. While every distro starts with the same foundational kernel, they can differ in package managers, default software, release cycles, desktop environments, and communities.

  Some distributions focus on user-friendliness and aim to provide a polished desktop experience suitable for beginners or non-technical users. Others cater to enterprise environments with stable, long-term support. Still others are built for advanced or minimalist users who value complete control over their systems. This diversity is one of Linux's greatest strengths, as it allows people to choose exactly what they need.

- **Popular Linux Distributions**

  There are **hundreds of Linux distros** available, each with its own unique approach. Here are a few well-known examples:

  - **Ubuntu**:
    Developed by Canonical, Ubuntu places a strong emphasis on user-friendliness and regular releases. It's available in both desktop and server editions, with Long-Term Support (LTS) versions that receive updates for five years.

- **Debian**:
  Known for its stability and conservative release cycle, Debian is the foundation for many other distros (including Ubuntu). Its large repository of packages makes it popular on servers and desktops alike.
- **Fedora**:
  A community-driven distro sponsored by Red Hat. Fedora offers the latest open-source software and acts as a testing ground for technologies that may eventually find their way into Red Hat Enterprise Linux (RHEL).
- **Arch Linux**:
  A "rolling release" distribution that provides cutting-edge software. Arch expects users to configure much of the system themselves, which can be a great learning experience but may be challenging for newcomers.
- **CentOS (Stream) / Rocky Linux / AlmaLinux**:
  These were historically derived from Red Hat Enterprise Linux (RHEL), focusing on stability for production environments. CentOS now follows a "Stream" model, while Rocky Linux and AlmaLinux aim to fill the traditional CentOS role as downstream rebuilds of RHEL.

Desktop vs. Server Editions

While some distros are offered in a single edition that you can customize for your needs, many provide distinct desktop and server variants. A desktop edition usually includes a graphical user interface (GUI), desktop applications (like a file manager, web browser, and office suite), and user-friendly tools. A server edition, on the other hand, typically installs minimal software by default, relying on command-line administration. This streamlined setup helps maintain security and reduce overhead.

Choosing the right edition depends on whether you plan to use Linux primarily for personal computing or for hosting services (like web servers, databases, or mail servers). You can, of course, convert a server edition into a desktop environment or vice versa by installing the needed packages.

Understanding Release Cycles

Different distros have different release cycles. For example, Ubuntu provides a stable LTS version every two years, while Arch Linux pushes continuous updates as part of its rolling release model. A stable, long-term release is often preferred in production servers and enterprise settings since major changes are infrequent and well-tested. In contrast, rolling release distros continuously deliver the latest software, which appeals to enthusiasts or developers who want the newest features and packages.

Installation Basics

Regardless of which distro you choose, the installation steps generally follow a similar pattern:

1. **Acquire the ISO Image**:
   You'll download an ISO file from the distro's official website. This ISO contains the entire operating system in a compressed form.
2. **Create a Bootable Media**:
   You can burn the ISO to a DVD or transfer it to a USB flash drive using tools like Rufus (on Windows) or Etcher (on multiple platforms). Most modern computers can boot from USB.
3. **Boot and Install**:
   - **Live Environment**: Many distros let you "try" Linux without installing, letting you test hardware compatibility and interface preferences.
   - **Full Installation**: You'll typically be guided through a setup wizard to partition your drive (or choose automatic partitioning), configure user accounts, and set regional settings.
4. **Set Up Post-Installation**:
   After installation completes, it's important to update the system by running commands like `apt update && apt upgrade` (Debian/Ubuntu/Mint) or equivalent commands (`dnf`, `pacman`, etc.) for other distros. Installing proprietary drivers (e.g., for graphics) may also be necessary depending on your hardware.

For new users, it's often recommended to install Linux on a virtual machine (VM) using software like VirtualBox or VMware Player. This approach eliminates the risks of overwriting your existing operating system and lets you experiment freely.

- ## Practice Exercise

1. Choose two Linux distros that interest you (for example, one desktop-focused distro like Ubuntu or Linux Mint, and one server-focused distro like Debian or CentOS).
2. Write a short paragraph comparing their target audiences, release cycles, and default software choices.
3. If you have a spare USB drive or prefer a VM, try creating a bootable USB or setting up a virtual machine for at least one of these distros.

   By comparing different distributions and, if possible, going through the initial steps of installation (either on real hardware or a VM), you'll develop a clearer sense of which distro best aligns with your goals and requirements.

Module 2: Basic Linux Commands

Lesson 2.1 – File System Navigation & Management

- **Lesson Objectives**
  - Navigate the Linux file system using essential commands.
  - Create, copy, move, and delete files and directories.
  - Manage permissions and ownership effectively.

- **Navigating the Linux File System**

The Linux file system is arranged in a tree-like hierarchy starting at the root directory, denoted by /. Under /, you'll find directories like /home, /usr, /etc, /var, and others that each serve distinct purposes. For instance, /home contains personal directories for individual users, while /etc holds system-wide configuration files.

To move around the file system efficiently, you'll rely on commands such as ls, cd, and pwd. The command pwd (print working directory) reveals the directory you're currently in. The ls command lists files and folders, with options like -l for a detailed view or -a to show hidden files. Changing directories is done with cd, followed by the desired path (e.g., cd /home/user or cd Documents if you're already in /home/user).

- **File and Directory Management**
  - Creating Files and Directories:
    - You can create an empty file using touch filename.txt.
    - To create a directory, use mkdir directory_name.
  - Copying, Moving, and Renaming:
    - Copy a file with cp source destination.
    - Move a file with mv source destination; if you specify a new filename, it effectively renames the file.
  - Removing Files and Directories:
    - Remove a file using rm filename.txt.
    - Remove a directory with rmdir directory_name, or rm -r directory_name to remove it along with any files inside.

Use caution with the rm -r command, as it recursively deletes everything within a directory.

Remember that on Linux, paths can be relative or absolute. An absolute path starts at the root directory (e.g., /home/user/Documents), while a relative path references the current directory (e.g., Documents/file.txt).

- **Permissions and Ownership**

  Linux enforces a security model based on ownership and permissions. Every file or directory has a user owner (usually the creator), a group, and permission bits for three categories: owner, group, and others. Permissions are typically represented by three sets of r (read), w (write), and x (execute).

  - **Changing Permissions:**
    - The chmod command can modify these permissions. For example, chmod u+x script.sh grants execute permission to the file owner.

- You can also use octal notation (e.g., chmod 755 file), where each digit represents owner, group, and others.

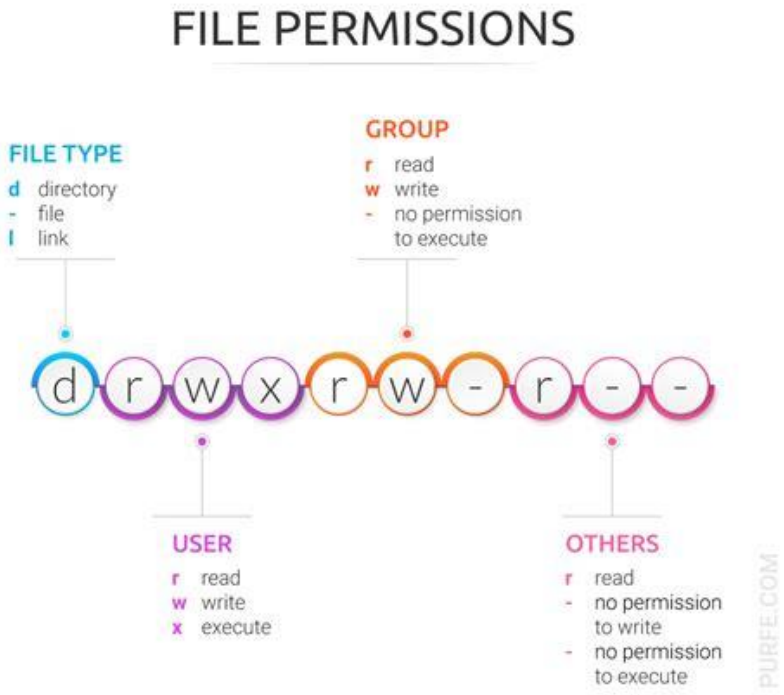
- **Changing Ownership:**
  - Use chown to change the owner: sudo chown new_owner file.txt.
  - chgrp adjusts the group ownership.

Properly setting permissions and ownership is crucial on multiuser systems, ensuring that only the right people (or processes) can modify or run certain files.

- **Practice Exercise**
  1. Create a directory named project inside your home directory.
  2. Inside project, create two files called notes.txt and data.csv using the touch command.
  3. Rename notes.txt to info.txt.
  4. Change the permissions of info.txt so only you (the owner) can read and write it.
  5. Verify the new permissions using ls -l.

By completing these steps, you'll practice essential file manipulation commands and see firsthand how permissions are reflected in the file system.

This tutorial guides beginners through the essentials of navigating the Linux file system. (Linux Commands for Beginners 04 - Navigating the Filesystem)



Lesson 2.2 – System Information, Piping, and Redirection

- **Lesson Objectives**
  - Retrieve system information to check resource usage.
  - Use pipes (|) and redirection (>, >>) to connect commands and manage output.
  - Filter and manipulate text with tools like grep, sort, and wc.

- **Viewing System Information**
  - Linux provides a variety of commands for checking system resource usage and status:
  - uname: Displays basic information about the operating system. Running uname -a shows the kernel version and other details.
  - df: Reports available disk space. Using df -h provides human-readable units, making disk usage easier to interpret.
  - free: Displays current memory usage. Adding -h here as well gives the output in megabytes or gigabytes.
  - top or htop: Shows running processes and system load in real-time, allowing you to monitor CPU and memory usage.

  Being familiar with these tools will help you keep tabs on system performance, particularly if you're running Linux on a server or workstation with resource-intensive applications.
- **Piping and Redirection**
  One of the most powerful aspects of Linux is the ability to chain commands together. When you use the pipe operator (|), you take the output of one command and feed it directly into another command as input.

  For example, ls -l /etc | grep ssh searches for items containing "ssh" within the /etc directory listing. This output can then be redirected to a file for later review by using > or >>.

    - Redirection operators:
      - >: Overwrites the target file with the command's output.
      - >>: Appends the command's output to the target file, preserving existing data.
      - <: Takes input from a file rather than standard input (less common in basic usage, but still valuable).

Using pipes and redirection allows you to quickly filter, sort, or store the results of a command without manually copying and pasting. This forms the basis of command-line automation and is a key building block toward shell scripting.

- **Useful Filtering Tools**
  - Besides using grep for text search, several built-in utilities can help process and manipulate text:
  - sort: Sorts lines alphabetically or numerically.
  - uniq: Removes consecutive duplicate lines (often paired with sort).

- wc: Counts lines, words, or characters. Using wc -l displays the number of lines in a file or piped output.
- head and tail: Show the first or last lines of a file or input stream.

  Combining these tools creatively can solve a wide range of text-processing tasks without having to open a graphical editor or write a script.

- **Practice Exercise**

  1. Run the command ls /etc | grep ssh to see if there are any files or directories related to ssh.
  2. Redirect the output of this command to a file named ssh_files.txt.
  3. Display how many lines are in ssh_files.txt using wc -l.
  4. If you'd like, explore the contents of ssh_files.txt using cat or less.

  This exercise will help you practice piping and redirection, as well as filtering command output with grep. By the end, you should feel more comfortable chaining commands together to produce meaningful results.

"Linux Terminal Basics | Navigate the File System on Ubuntu" – This video covers fundamental commands, including those for system information and managing command output. (Linux Terminal Basics | Navigate the File System on Ubuntu)

Module 3: Application and Exercises (Commands)

Lesson 3.1: Hands-On Mini-Projects

- **Lesson Objectives**
  - By the end of this lesson, you should be able to:
  - Apply fundamental Linux commands in practical scenarios.
  - Create small scripts or command sequences to automate common tasks (without advanced scripting features).
  - Understand how to combine multiple commands to achieve real-world goals.

- **Reinforcing Basic Commands Through Practical Exercises**

  - Learning commands in isolation is valuable, but putting them into practice is where you truly solidify your understanding. This lesson focuses on mini-projects that combine file navigation, permissions, and text-processing commands. You'll simulate real-world tasks such as searching for files, creating backups, and organizing data in a structured way.

  - **Mini-Project #1: System Report (Command-Based)**

    A system report is a quick snapshot of relevant system metrics—such as disk usage, memory usage, and running processes. While full scripting comes later, you can still chain together commands to generate a simple report.

- Suggested Steps:
  - Gather disk usage with df -h.
  - Display current memory usage with free -h.
  - List top five processes by CPU or memory usage using ps aux --sort=-%mem | head -n 6.
  - Combine these outputs (using piping and redirection) into a text file named system_report.txt.

  By creating this report entirely from the command line, you'll see how multiple utilities can work together to provide insight into system health—an essential skill for both desktop users and system administrators.

- **Mini-Project #2: File Search & Organization**

  Another common task is locating specific files and automatically copying or archiving them for later use. This might involve log files, images, or any file that follows a certain naming pattern.

  - Suggested Steps:
    - Use the find command to locate files of a certain type or within a given date range (for instance, find /var/log -type f -name "*.log").
    - Pipe the results into a command like xargs cp or cp in a while loop to copy these files into a designated folder—e.g., /home/user/log_archive.
    - Verify the presence of each file in the log_archive folder by listing its contents (ls log_archive).

    This exercise reinforces your mastery of searching, copying, and organizing files. You can also choose to compress them (e.g., tar -czf logs_archive.tar.gz log_archive) if you're comfortable with tar and gzip commands.

- **Practice Exercise**
  1. Create a commands-only "mini script" (a sequence of commands in your terminal or a short .sh file, if you wish) to generate a system report.
  2. Then, pick a file type on your system (like .txt or .png) and automate moving all such files from one directory to another for organization.
  3. Optional: Compress the resulting folder into a .tar.gz archive.

  By chaining commands, you'll see how quickly you can transform scattered data into a consolidated, meaningful output—all with just the terminal.

Lesson 3.2 – Deeper into Permission Management

- **Lesson Objectives**
  - Configure complex file permissions and special directory attributes.
  - Explain how user groups and sticky bits function in multi-user environments.

- Implement secure folder sharing scenarios.

- **Advanced Permissions Concepts**

  While you've already encountered basic permissions-read (r), write (w), and execute (x)-Linux offers additional capabilities for more nuanced control:
  - Sticky Bit: Commonly used on shared directories (e.g., /tmp), the sticky bit ensures that only the owner of a file (or root) can delete or rename the file, even if others have write permissions to that directory.
  - Setgid: When applied to a directory, new files created within it inherit the directory's group ownership instead of the creating user's default group. This is useful for collaborative projects where files must consistently belong to the same group.

  Understanding and correctly applying these features helps maintain a secure yet cooperative environment, especially in multi-user setups.

- **Group Management**

  Linux allows you to classify users into groups, making it easier to manage collective permissions. For instance, you might have a developers group that needs write access to project files, while the guests group might only read those files.

  - Creating a Group: Use sudo groupadd group_name.
  - Adding Users: sudo usermod -a -G group_name username.
  - Verifying Group Membership: Check a user's groups by running groups username.

  By leveraging groups effectively, you avoid the overhead of assigning permissions to individual users for each resource. Instead, you configure group-level access once and simply add or remove users as needed.

- **Practical Scenarios**

  Imagine a situation in which several team members need to collaborate on documents in a shared folder but should not alter each other's files. You can create a shared_data folder, assign it a specific group, enable the sticky bit to prevent accidental file deletions, and adjust permissions so that each user can write their own files without risking others' data.
  - Sticky Bit Example:
    I. Create a directory: mkdir /home/shared_data.
    II. Assign a group: sudo chgrp devteam /home/shared_data.
    III. Set directory permissions and sticky bit: chmod 1770 /home/shared_data.
      - 1 sets the sticky bit,
      - 7 grants rwx to the owner,
      - 7 grants rwx to the group,
      - 0 denies all access to others.

Now users in the devteam group can create files, yet only the file owner (or root) can delete them.

- **Practice Exercise**
  - Create a new group named project_team.
  - Assign at least two different user accounts to that group (you may need to create additional user accounts if you only have one).
  - Create a folder named team_folder, set its group to project_team, and apply permissions so everyone in project_team has read/write access.
  - Enable the sticky bit on team_folder to prevent users from deleting one another's files.

  This exercise simulates a multi-user, collaborative environment and challenges you to apply advanced permissions and group management tools in a practical way.

- **Module 4: Introduction to Shell Scripting**

  Lesson 4.1 – What Is Shell Scripting?

  - **Lesson Objectives**
    - By the end of this lesson, you should be able to:
    - Define what a shell script is and why it's useful.
    - Understand the role of the Bash shell (or other shells) in running commands.
    - Create and execute a simple "Hello World" script.

    This introductory video explains the fundamentals of shell scripting. (Shell Scripting Tutorial for Beginners 1 - Introduction)

  - **The Concept of Shell Scripting**

    A shell script is essentially a text file containing a series of commands that the shell (e.g., Bash) can run sequentially. Instead of entering each command interactively at the terminal, you write them into a script that can be executed, thus automating repetitive or complex tasks. This allows you to save time and reduce the likelihood of human error.

    Linux systems typically include several shells, but the Bash shell (Bourne-Again Shell) is the most common default. Other popular shells include Zsh, Fish, and Dash, each with unique features and syntax nuances. For this course, we'll focus primarily on Bash because of its widespread availability and extensive documentation.

- **Core Components of a Shell Script**
  - When you open a shell script in a text editor, you'll notice a few key elements:
    - Shebang Line: Usually, the first line in a script, it tells the system which interpreter to use. Commonly you'll see:
      - #!/usr/bin/env bash *or* #!/bin/bash
    - Commands: The same commands you'd enter at the terminal (e.g., ls, echo, or pwd).
    - Comments: Lines starting with # (other than the shebang) are ignored by the shell and serve as explanations or notes for anyone reading the script.
    - Executable Permissions: To run a script directly (e.g., ./script.sh), you need to make it executable with chmod +x script.sh.

    By combining these elements, you create a file that the shell can interpret step by step, executing each command in turn.

- **Why Automate with Shell Scripting?**

  Automation is the heart of shell scripting. Whether you need to manage file backups, monitor system resources, or deploy software updates, a script can handle each step systematically, reducing manual input. This is especially crucial in server administration, DevOps, and continuous integration environments, where reliability and consistency are paramount.

  Shell scripting also acts as a foundation for more advanced scripting or programming languages. Many principles you learn in Bash (like variables, conditionals, and loops) will transfer to languages such as Python or Perl, making shell scripting an excellent starting point for broader automation skills.

- **Practice Exercise**
  - Open a text editor (e.g., nano, vi, or gedit) and create a file named hello_world.sh.
  - At the top, include the shebang line: #!/usr/bin/env bash
  - Add a simple echo command, such as echo "Hello, World!"
  - Save the file, then run chmod +x hello_world.sh to make it executable.
  - Execute it with ./hello_world.sh and observe the output.

This will be your first shell script, illustrating how quickly you can turn a few terminal commands into an automated process.

Lesson 4.2 – Variables, Conditionals, and Loops

- **Lesson Objectives**
  - By the end of this lesson, you should be able to:
  - Create and use variables in a Bash script.
  - Implement conditional statements (if, elif, else).
  - Write loops (for, while) to iterate over data or repetitive tasks.

  This comprehensive tutorial covers variables, conditionals, and loops in shell scripting. (Shell Scripting Crash Course | Linux Certification Training | Edureka)

- **Variables in Shell Scripting**
  - Variables allow you to store values or data that can be referenced throughout the script. Defining a variable is as simple as:

```
myvar="Hello"
echo $myvar
```

  Notice there is no space around the equals sign in variable assignments. You can reference the variable by prefixing its name with `$`. Bash variables typically hold strings, although you can also store integers for arithmetic operations by using the built-in `let`, `$(( ))`, or `expr` commands.

- Conditional Statements
  - Conditional statements let your script branch and make decisions based on certain criteria. An if statement has the general form:

```
if [ condition ]; then
  # commands if condition is true
elif [ another_condition ]; then
  # commands if the second condition is true
else
  # commands if neither condition is true
Fi
```

  Conditions can be:

  String comparisons: ["$var" = "Hello"]

  Integer comparisons: [$num -gt 10] (greater than)

File checks: [-f filename] (checks if a file exists)

When combined with command exit codes (e.g., if [$? -eq 0] for success), conditionals let you build robust error-checking and workflow control in your scripts.

- **Loops**

  Loops help you repeat commands for a set of items or while a condition is met. The two most common loop types in Bash are:

  - For Loops:

```
for file in *.txt
do
  echo "Processing $file"
done
```

  This loop iterates over each .txt file in the current directory, performing actions on each file.

  - **While Loops**:

```
while [ some_condition ]
do
  # commands
Done
```

  This loop continues running as long as the condition remains true. It's often used for reading from input streams or waiting until a particular event occurs.

Combining loops with conditionals allows for complex logic, such as iterating over a folder of files and skipping those that don't match your criteria.

- **Practice Exercise**
  - Write a script named cleanup.sh.
  - Inside, define a variable TARGET_DIR="/path/to/folder" (replace with a real path).
  - Use a for loop to iterate over all .tmp files in TARGET_DIR. For each file, check if it exists and then delete it if it's older than a certain number of days. (You can use find or an if statement to decide if the file is old.)

- Print a message each time a file is removed.

By creating cleanup.sh, you'll gain hands-on experience with variables, loops, and conditionals, reinforcing how to combine them for real-world automation tasks.

Lesson 4.3 – Functions and Basic Debugging

- **Lesson Objectives**
  - Define and call functions in a Bash script.
  - Organize longer scripts into modular sections using functions.
  - Debug scripts using built-in Bash options like -x and trap.

- **Functions for Script Organization**
  As your scripts grow more complex, functions let you group logically related commands under a single name. This makes your code more readable and easier to maintain. You can define a function in Bash in various ways:
  - php:

```
function my_function() {
  echo "This is my function"
}
```

  - javascript:

```
my_function() {
  echo "This is my function"
}
```

  Once defined, you call the function by writing its name, just as if it were a built-in command. Functions can also accept parameters (e.g., $1, $2) for flexibility. Organizing your script into functions that handle different tasks such as collecting system data or cleaning up temporary files helps prevent your code from becoming a monolithic block of commands.

- **Basic Debugging Techniques**
  - Shell scripting can be error-prone, especially for newcomers. Fortunately, Bash offers tools to help you identify and fix issues quickly:
  - -x (Execution Trace): Running bash -x script.sh prints each command as it executes, revealing the flow of your script and showing how variables are expanded.
  - -e (Exit on Error): Adding set -e at the top of your script forces the script to exit immediately if any command fails, preventing further commands from running in an invalid state.

- trap: This mechanism lets you catch signals (like SIGINT or script errors) and run specific commands before the script exits. You might use trap to clean up temporary files or reset permissions if something goes wrong.

Debugging often involves systematically isolating which part of the script is failing by adding echoes or removing suspicious lines. Over time, you'll develop an instinct for typical shell pitfalls, such as unquoted variables or misplaced brackets.

- **Putting It All Together: sysinfo.sh**
  As a culminating example, consider writing sysinfo.sh to gather system metrics. Here's a rough outline
  - **Functions:**
    - check_disk(): runs df -h
    - check_memory(): runs free -h
    - check_users(): runs who
  - **Main Body:**
    - Calls each function in sequence to produce a neat summary.
    - Optionally, uses bash -x sysinfo.sh to see the script's flow for debugging.

  By breaking each task into a function, you keep your script organized and easily extend it with new features or checks in the future.
- Practice Exercise (Lesson 4.3)
  - Create a script called sysinfo.sh.
  - Define three functions: show_disk(), show_memory(), and show_users().
  - Each function runs the relevant command (e.g., df -h, free -h, who) and echoes a short header before printing the results.
  - In the main script, call these functions in a logical order (e.g., disk → memory → users).
  - Run the script with bash -x sysinfo.sh and observe the command trace.

  This exercise helps you practice function-based organization and debugging. By examining how each function executes, you'll gain confidence in your ability to modularize scripts and troubleshoot issues that arise.

Module 5: Applications and Exercises (Shell Scripting)

Lesson 5.1 – Real-World Shell Scripting Scenarios

- **Lesson Objectives**
  - By the end of this lesson, you should be able to:

- Build shell scripts that automate practical, real-world tasks.
- Integrate user input and system feedback in your scripts.
- Schedule scripts to run automatically (intro to cron).

- **Real-World Automation with Shell Scripts**
  While you can use shell scripting for simple tasks, its true power emerges when you apply it to everyday system administration or DevOps scenarios. In this lesson, we'll explore scripts that back up data, monitor system resources, and even interact with users. The common thread among these scenarios is consistent, repeatable automation—scripts take commands you already know and chain them into efficient, robust workflows.

  - **Backup and Archive Scripts**
    - Backing up data regularly is one of the most common reasons to automate tasks. A well-structured backup script can:
    - Identify which files or directories need to be backed up (e.g., /home/username/Documents).
    - Compress them using tools like tar, gzip, or zip.
    - Store the archive in a safe location, whether it's a local folder, external drive, or cloud service.

  A basic approach might involve creating a script called backup.sh that uses tar -czf backup.tar.gz /path/to/data. Once the script is verified, you can schedule it with cron to run automatically (e.g., nightly or weekly). This ensures your important files are consistently backed up without manual intervention.

- **Monitoring & Alerting Script**
  - In a professional environment, keeping tabs on system health is critical. A script can:
    - Check CPU usage using top or ps
    - Review available disk space using df -h.
    - Scan log files for errors using grep.
    - Send alerts (via email or syslog) if usage crosses a defined threshold.

  For instance, you could define a script named monitor.sh that checks CPU usage, compares it to a set limit (like 80%), and sends a warning if exceeded. Similarly, you might track memory usage or disk space, alerting you when free space falls below a certain percentage. This type of proactive monitoring script can prevent system crashes and minimize downtime.
- User Interaction

Scripts can prompt users for input to make them more flexible and interactive. The Bash built-in command read enables you to collect user data, which you can then use within conditionals or loops. For example:

```
echo "Enter the directory to back up:"
read backup_dir
echo "Compressing $backup_dir ..."
tar -czf backup.tar.gz "$backup_dir"
```

This ensures the script isn't limited to one specific directory or file. Additionally, you can prompt users for confirmation, ask them to select from options, or even capture command-line arguments if you prefer a more standard approach to user input.

- Practice Exercise (Lesson 5.1)
    - **Create a script named monitor.sh that:**
        - Asks the user to enter a CPU usage threshold (e.g., 80%).
        - Collects current CPU usage from a command like top -b -n1 | grep "Cpu(s)"
        - Compares the current usage to the threshold.
        - Displays an alert message if usage exceeds that limit.
    - **(Optional) Extend the script to check memory usage as well, printing a similar warning if memory usage is too high.**

By integrating user interaction with monitoring, you gain experience writing scripts that can adjust to different situations and user preferences.